POLITECHNIKA WROCŁAWSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

PROJEKTOWANIE EFEKTYWNE ALGORTYMÓW

TSP – Problem Komiwojażera

Zadanie 1

Grupa: Wtorek 9:15

Autor:

Karolina Obiedzińska

Indeks: 273034

Prowadzący:

dr inż. Marcin Łopuszyński

Spis treści

1. Wstęp 3

1.1 Opis rozpatrywanego problemu 3

1.2. Oszacowanie złożoności problemu 3

2. Przykład praktyczny 4

3. Plan eksperymentu 4

4. Wyniki eksperymentu 6

4.1 Macierze symetryczne 6

4.2 Macierze asymetryczne 9

5. Wnioski 13

6. Kod źródłowy 14

Bibliografia: 14

1. Wstęp

1.1 Opis rozpatrywanego problemu

Problem komiwojażera[1](ang. Traveling Salesman Problem, TSP) to jedno z klasycznych zagadnień optymalizacyjnych w teorii grafów i informatyce. Polega na znalezieniu najkrótszej możliwej trasy, która pozwala komiwojażerowi odwiedzić każde z zadanych miast dokładnie raz i wrócić do miasta startowego. Jest to problem NP-trudny, co oznacza, że nie istnieje znany algorytm, który rozwiązuje go w czasie wielomianowym dla dużych zbiorów danych. Z tego powodu opracowano różne podejścia i heurystyki do jego rozwiązywania.

Algorytmy rozwiązujące problem komiwojażera:

- **Algorytm przeglądu zupełnego (brute-force)**
  W podejściu brute-force generujemy wszystkie możliwe trasy (permutacje miast), a następnie wybieramy tę, której koszt jest najmniejszy.

- **Algorytm najbliższego sąsiada (nearest neighbor)**
  Heurystyczne podejście, które polega na tym, że w każdym kroku komiwojażer wybiera do odwiedzenia najbliższe jeszcze nieodwiedzone miasto[2]. Algorytm ten nie gwarantuje optymalnego rozwiązania, ale jest szybki i prosty w implementacji.

- **Algorytm losowy**
  W tym podejściu rozwiązanie generowane jest poprzez losowy wybór kolejności odwiedzanych miast. Chociaż algorytm ten jest bardzo szybki i łatwy do wdrożenia, jego skuteczność w znalezieniu dobrego rozwiązania jest niewielka.

1.2. Oszacowanie złożoności problemu

- **Algorytm przeglądu zupełnego**
  Algorytm ten charakteryzuje się wyjątkowo wysoką złożonością obliczeniową — O(n!) w przypadku przeszukiwania wszystkich możliwych ścieżek lub O((n-1)!) przy pominięciu cyklicznych tras, gdzie n oznacza liczbę miast. [3]

- **Algorytm najbliższego sąsiada**
  Jego złożoność obliczeniowa wynosi $O(n^2)$, co czyni go efektywnym dla średniej wielkości problemów, jednak wynik trasy może być daleki od optymalnego.

- **Algorytm losowy**
  Ze względu na losowość działania tego algorytmu, trudno precyzyjnie określić jego złożoność obliczeniową. Niemniej, algorytm ten nie gwarantuje uzyskania optymalnego rozwiązania, a jego efektywność jest trudna do przewidzenia w konkretnych przypadkach.

---

[1] https://pl.wikipedia.org/wiki/Problem_komiwoja%C5%BCera
[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein - *"Introduction to Algorithms"*
[3] https://tonicanada.medium.com/introduction-to-the-travelling-salesman-problem-5ace44932cb5

2. Przykład praktyczny

- **Algorytm najbliższego sąsiada[4] – dla macierzy symetrycznej**
  Algorytm działa w sposób "chciwy", rozpoczynając od każdego wierzchołka i próbując znaleźć najkrótszą możliwą trasę.
  Dla N=3

$$C=\begin{bmatrix} -1 & 10 & 15 \\ 10 & -1 & 20 \\ 15 & 20 & -1 \end{bmatrix}$$

Rysunek 1 Przykładowa macierz
Źródło: Opracowanie własne

Iteracja 1 (start z miasta 0):
Zaczynamy w mieście 0. Najmniejszy koszt przejścia to 10 do miasta 1.
Z miasta 1 wybieramy miasto 2 (koszt 20).
Powrót z miasta 2 do 0 kosztuje 15. Całkowity koszt = 45.
Iteracja 2 (start z miasta 1):
Zaczynamy w mieście 1. Najmniejszy koszt to 10 do miasta 0.
Z miasta 0 wybieramy miasto 2 (koszt 15).
Powrót z miasta 2 do 1 kosztuje 20. Całkowity koszt = 45.
Iteracja 3 (start z miasta 2):
Zaczynamy w mieście 2. Najmniejszy koszt to 15 do miasta 0.
Z miasta 0 wybieramy miasto 1 (koszt 10).
Powrót z miasta 1 do 2 kosztuje 20. Całkowity koszt = 45.
Minimalny koszt trasy dla wszystkich startów wynosi 45.

- **Algorytm losowy**
  Program losowo generuje trasy dla problemu komiwojażera (TSP), tasując kolejność miast i obliczając koszt każdej trasy. Działa w pętli, porównując wynik z minimalnym kosztem, dopóki nie znajdzie trasy nieznacznie gorszej niż koszt brute-force (maks. 1.25 razy wyższy) lub osiągnie limit 1000 prób. Zwraca najmniejszy znaleziony koszt trasy.

3. Plan eksperymentu

3.1. Rozmiary używanych struktur danych

Będą generowane wyniki dla 7 reprezentatywnych rozmiarów macierzy: 5,6,7,8,9,10,11. Macierze będą symetryczne i asymetryczne.

Macierz jest przechowywana w obiekcie klasy Matrix. Obiekt posiada atrybuty rozmiar, wskaźnik do tablicy dwuwymiarowej z wartościami z macierzy oraz informacje czy macierz jest symetryczna, czy asymetryczna.

[4] http://algorytmy.ency.pl/artykul/algorytm_najblizszego_sasiada

3.2. Sposób generowania danych

Macierz możemy wygenerować na dwa sposoby: odczytanie z pliku oraz wygenerowanie macierzy losowej.

- *Odczytanie z pliku:* W tym przypadku wystarczy podać nazwę pliku znajdującego się w katalogu „Aplikacja/FilesToRead”. Macierz zostanie odczytana z tego pliku.
- *Generowanie macierzy losowej:* Wymaga podania rozmiaru macierzy. Powstała macierz będzie dynamiczną, dwuwymiarową tablicą wypełnioną wartością „-1”. Następnie musimy wypełnić macierz losowymi wartościami. Jeśli chcemy, aby macierz była asymetryczna, używamy funkcji „fillRandom()”. Dla macierzy symetrycznej korzystamy z funkcji „fillRandomSymmetric()”.

3.3. Metoda pomiaru czasu

W celu pomiaru czasu wykonania algorytmu użyto funkcji QueryPerformanceCounter() oraz QueryPerformanceFrequency(), które umożliwiają uzyskanie dokładnego wyniku dzięki wysokiej precyzji dostępnej w systemach Windows. Pomiary zostały wykonane w mikrosekundach.

Działanie funkcji jest następujące:

1. Na początku uruchamiany jest stoper (startTimer()) za pomocą funkcji QueryPerformanceCounter(), która zapisuje aktualną wartość licznika wydajności do zmiennej „start”.
2. Po zakończeniu operacji, stoper jest zatrzymywany (stopTimer()) i zapisywana jest wartość „end”.
3. Funkcja diffTime() oblicza różnicę pomiędzy wartościami „start” i „end”, aby uzyskać czas wykonania operacji. W tym celu wykorzystywana jest także funkcja QueryPerformanceFrequency(), która podaje częstotliwość licznika.
4. Różnica difference.QuadPart jest przeliczana na mikrosekundy przez pomnożenie przez 1 000 000 i podzielenie przez częstotliwość.

Podczas testowania zapisuje się czas rozpoczęcia działania, a po zakończeniu wykonania obliczana jest różnica czasu. Czas ten jest sumowany w zmiennej „sred” w celu obliczenia średniego czasu działania dla kilku iteracji.

3.4. Warunki badawcze:

a. specyfikacja sprzętu:
   i. procesor: AMD Ryzen 7 7840HS Radeon 780M Graphics
   ii. zegar: 3801 Mhz
   iii. wielkość pamięci RAM: 32 GB
b. Badania były przeprowadzone na prezentowanym sprzęcie. Komputer pracował na zasilaniu sieciowym. Proces nie był przypisany do żadnego z rdzeni procesora, ani nie miał ustalonej częstotliwości taktowania.

4. Wyniki eksperymentu

4.1 Macierze symetryczne

Tabela 1 Tabela pokazuje czasy wykonania trzech różnych algorytmów rozwiązujących problem komiwojażera (TSP) dla różnych wielkości instancji n miast dla macierzy symetrycznych.

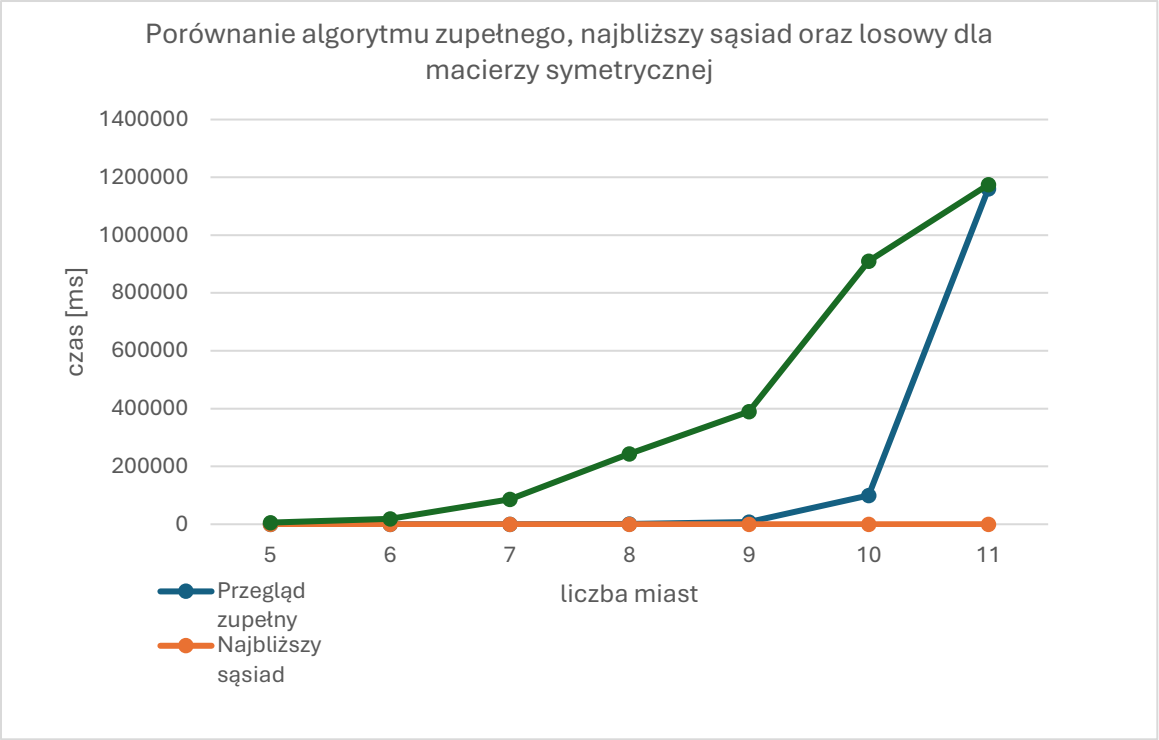
Symetryczne			
	Przegląd zupełny	Najbliższy sąsiad	Losowy
n	**Czas [ms]**		
5	6	12	5711
6	29	16	18714
7	186	25	86522
8	1349	36	243239
9	8092	41	390232
10	100098	74	910153
11	1160145	126	1174571

Źródło: Opracowanie własne

Algorytm przeglądu zupełnego, mimo gwarantowania rozwiązania optymalnego, charakteryzuje się wykładniczym wzrostem czasu obliczeń w miarę zwiększania liczby miast. Widać to szczególnie przy n = 10 i n = 11, gdzie czas wykonania gwałtownie rośnie, przekraczając nawet milion milisekund.

Algorytm najbliższego sąsiada, choć mniej precyzyjny, wykazuje znacznie lepszą wydajność czasową — czas wzrasta liniowo, co czyni go bardziej praktycznym dla średnich instancji problemu. Przykładowo, dla n = 11, czas jego działania to 126 ms, co jest nieporównywalnie krótsze w porównaniu do algorytmu przeglądu zupełnego.

Algorytm losowy, jak można się spodziewać, charakteryzuje się zmiennym i często bardzo długim czasem wykonania, co czyni go nieprzewidywalnym. Przy n = 11 czas jego działania wynosi ponad milion milisekund, co wskazuje na jego niewydolność przy większych instancjach problemu.



Rysunek 2 Rysunek pokazuje czasy wykonania trzech różnych algorytmów rozwiązujących problem komiwojażera (TSP) dla różnych wielkości instancji n miast dla macierzy symetrycznych.
Źródło: Opracowanie własne

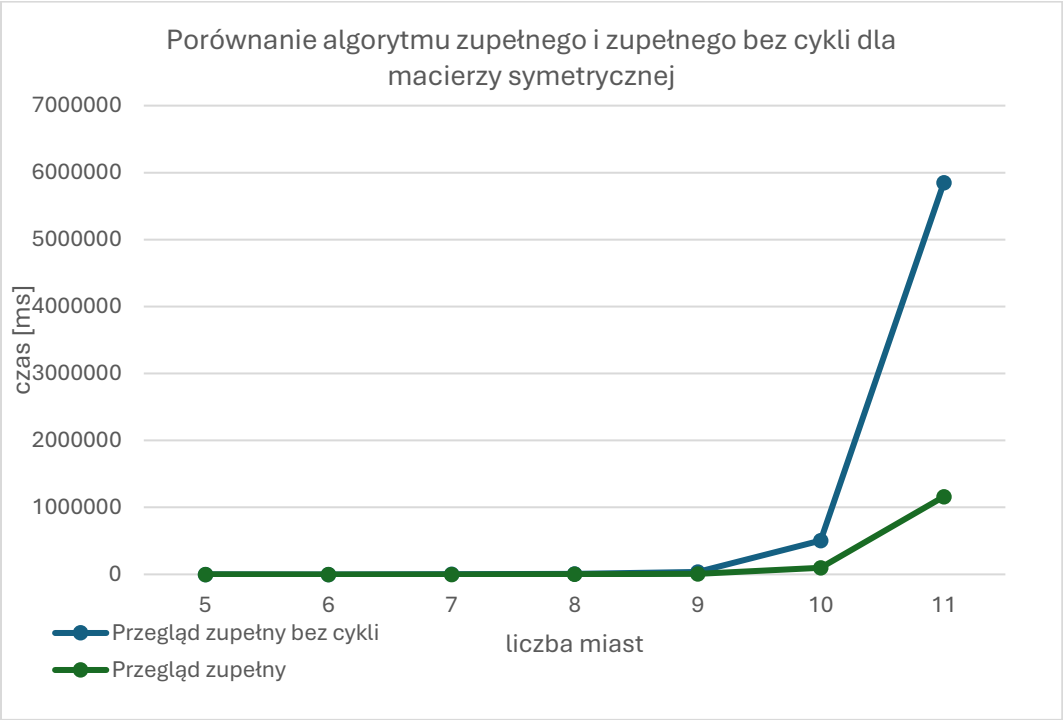
Tabela 2 Tabela porównuje czasy wykonania dwóch wariantów algorytmu przeglądu zupełnego: klasycznego oraz bez uwzględniania ścieżek cyklicznych, w rozwiązaniu problemu komiwojażera (TSP) dla różnych liczb miast (n) w przypadku macierzy symetrycznych.

	Symetryczne	
	Przegląd zupełny	Przegląd zupełny bez cykli
n	**Czas [ms]**	
2	0	2
3	0	3
4	1	8
5	6	28
6	29	128
7	186	813
8	1349	6045
9	8092	36808
10	100098	503865
11	1160145	5850052

Źródło: Opracowanie własne

Wyniki pokazują, że czas działania obu algorytmów wzrasta wykładniczo wraz ze wzrostem liczby miast, jednak wariant bez cykli potrzebuje znacznie więcej czasu, co wynika z większej liczby operacji do wykonania. Dla małych wartości n (np. 2 lub 3) różnice są minimalne, natomiast przy większych wartościach n, różnice stają się znaczące. Przykładowo, dla n = 10, algorytm przeglądu zupełnego potrzebuje około 100 sekund, natomiast jego wariant bez cykli aż około 504 sekund.

Dla n = 11 czas działania obu algorytmów gwałtownie wzrasta. Klasyczny algorytm wymaga około 1,16 miliona milisekund, natomiast wariant bez cykli aż 5,85 miliona milisekund, co oznacza, że różnica czasowa staje się ponad pięciokrotna.



Rysunek 3 Porównanie czasów wykonania dwóch wariantów algorytmu przeglądu zupełnego: klasycznego oraz bez uwzględniania ścieżek cyklicznych, w rozwiązaniu problemu komiwojażera (TSP) dla różnych liczb miast (n) w przypadku macierzy symetrycznych.
Źródło: Opracowanie własne

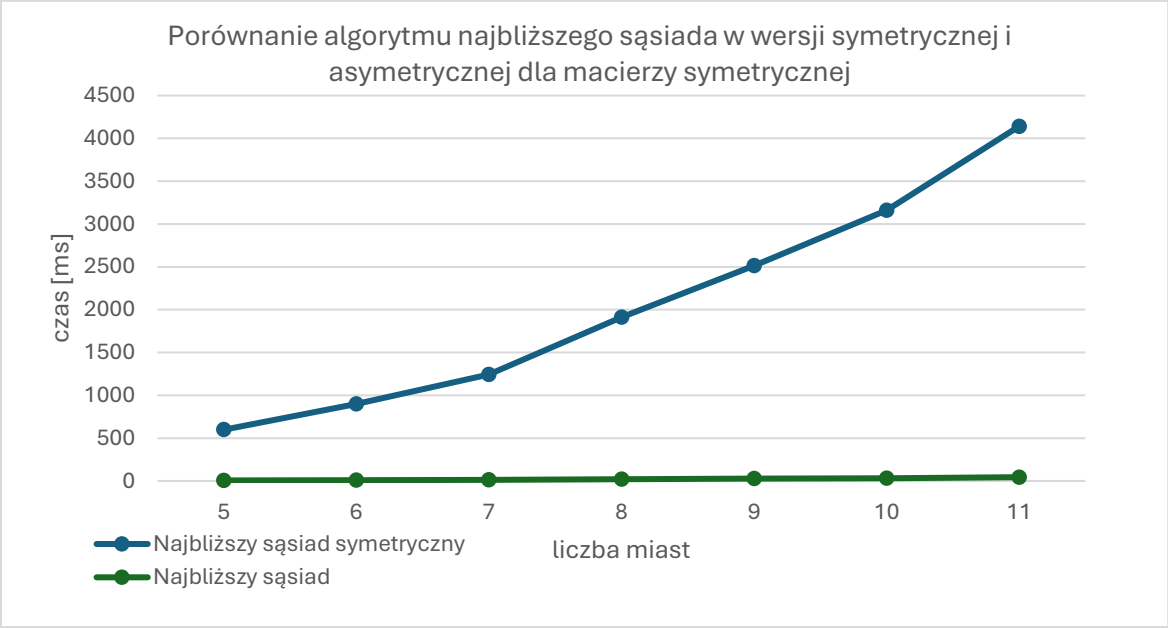
Tabela 3 Tabela porównuje czasy wykonania dwóch wariantów algorytmu najbliższego sąsiada dla różnych wielkości instancji problemu komiwojażera (TSP), gdzie n oznacza liczbę miast: klasycznego algorytmu najbliższego sąsiada oraz symetrycznego.

	Symetryczne	
	Najbliższy sąsiad	Najbliższy sąsiad symetryczny
n	**Czas [ms]**	
2	1	101
3	1	167
4	3	312
5	6	598
6	9	900
7	13	1241
8	20	1913
9	26	2513
10	33	3160
11	44	4140

Źródło: Opracowanie własne

Z analizy wyników można zauważyć, że czas działania obu algorytmów wzrasta wraz ze wzrostem liczby miast, jednak algorytm najbliższego sąsiada symetryczny jest

zdecydowanie bardziej czasochłonny. Różnice w czasie stają się zauważalne już dla n = 2, gdzie czas dla klasycznego algorytmu wynosi 1 ms, podczas gdy dla wariantu symetrycznego wynosi 101 ms. Ta tendencja utrzymuje się również przy większych wartościach n — dla n = 11 klasyczny algorytm potrzebuje 44 ms, podczas gdy algorytm symetryczny aż 4140 ms.



Rysunek 4 Porównanie czasów wykonania dwóch wariantów algorytmu najbliższego sąsiada dla różnych wielkości instancji problemu komiwojażera (TSP), gdzie n oznacza liczbę miast: klasycznego algorytmu najbliższego sąsiada oraz symetrycznego.
Źródło: Opracowanie własne

4.2 Macierze asymetryczne

Tabela 4 Tabela pokazuje czasy wykonania trzech różnych algorytmów rozwiązujących problem komiwojażera (TSP) dla różnych wielkości instancji n miast dla macierzy asymetrycznych.

Asymetryczne			
	Przegląd zupełny	Najbliższy sąsiad	Losowy
n	**Czas [ms]**		
2	0	1	291
3	0	3	162
4	2	7	486
5	7	15	14712
6	45	19	41116
7	201	24	163621
8	1400	34	525829
9	8334	37	618528
10	76990	54	731137
11	1204600	127	1148971

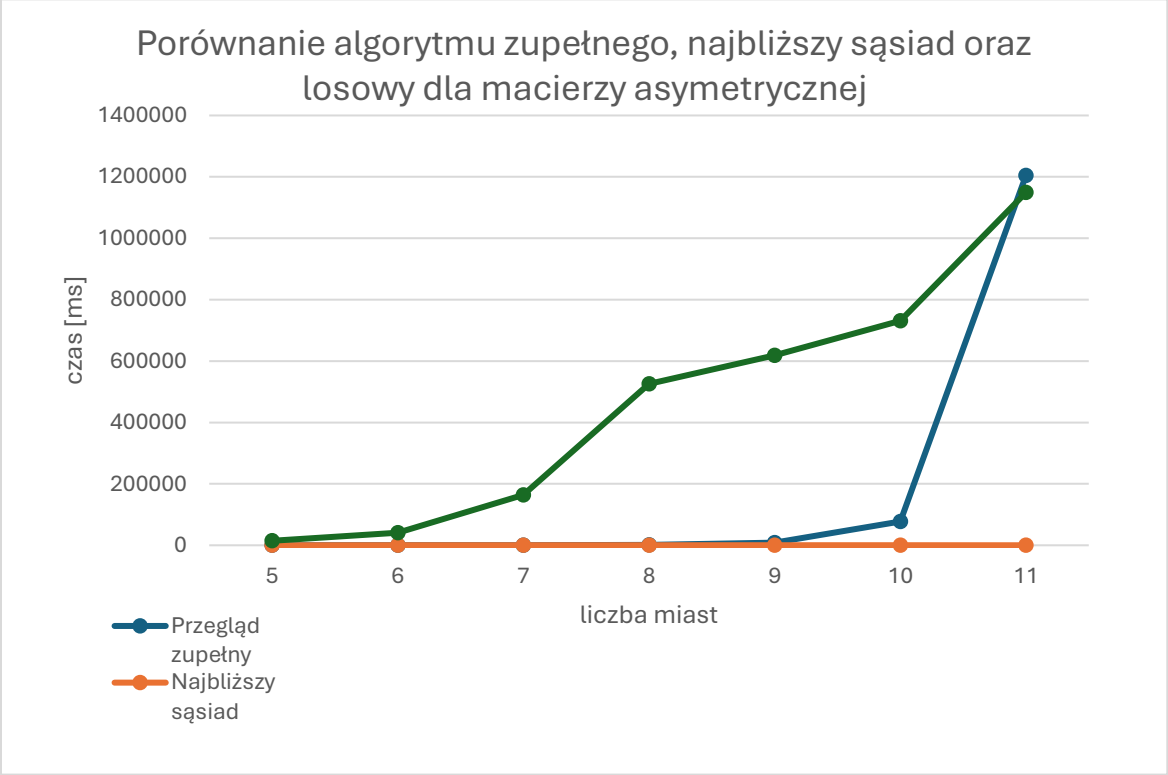
Źródło: Opracowanie własne

Algorytm przeglądu zupełnego gwarantuje znalezienie optymalnego rozwiązania, jednak jego czas działania rośnie wykładniczo wraz ze wzrostem liczby miast. Dla

małych wartości n (np. 2 lub 3) czas ten jest znikomy, ale już przy n = 10 wynosi około 77 sekund, a przy n = 11 przekracza 1,2 miliona milisekund. Wzrost ten ukazuje ograniczenia algorytmu w przypadku większych instancji problemu, gdzie jego zastosowanie staje się niepraktyczne.

Algorytm najbliższego sąsiada jest znacznie szybszy niż przegląd zupełny. Jego czas działania wzrasta wolniej, co czyni go bardziej efektywnym dla średnich i większych instancji problemu. Dla n = 11 czas wynosi zaledwie 127 ms, co w porównaniu do przeglądu zupełnego pokazuje jego dużą przewagę pod względem szybkości, choć kosztem precyzji rozwiązania.

Algorytm losowy, ze względu na swój charakter, wykazuje duże wahania w czasie działania. Dla małych wartości n (np. 2–4) czasy te są zbliżone do pozostałych algorytmów, ale dla większych instancji, takich jak n = 5 lub więcej, czas działania gwałtownie rośnie, osiągając ponad milion milisekund dla n = 11. Jest to wynik zmienności tego algorytmu i braku systematyczności w przeszukiwaniu przestrzeni rozwiązań.



Rysunek 5 Rysunek pokazuje czasy wykonania trzech różnych algorytmów rozwiązujących problem komiwojażera (TSP) dla różnych wielkości instancji n miast dla macierzy asymetrycznych.
Źródło: Opracowanie własne

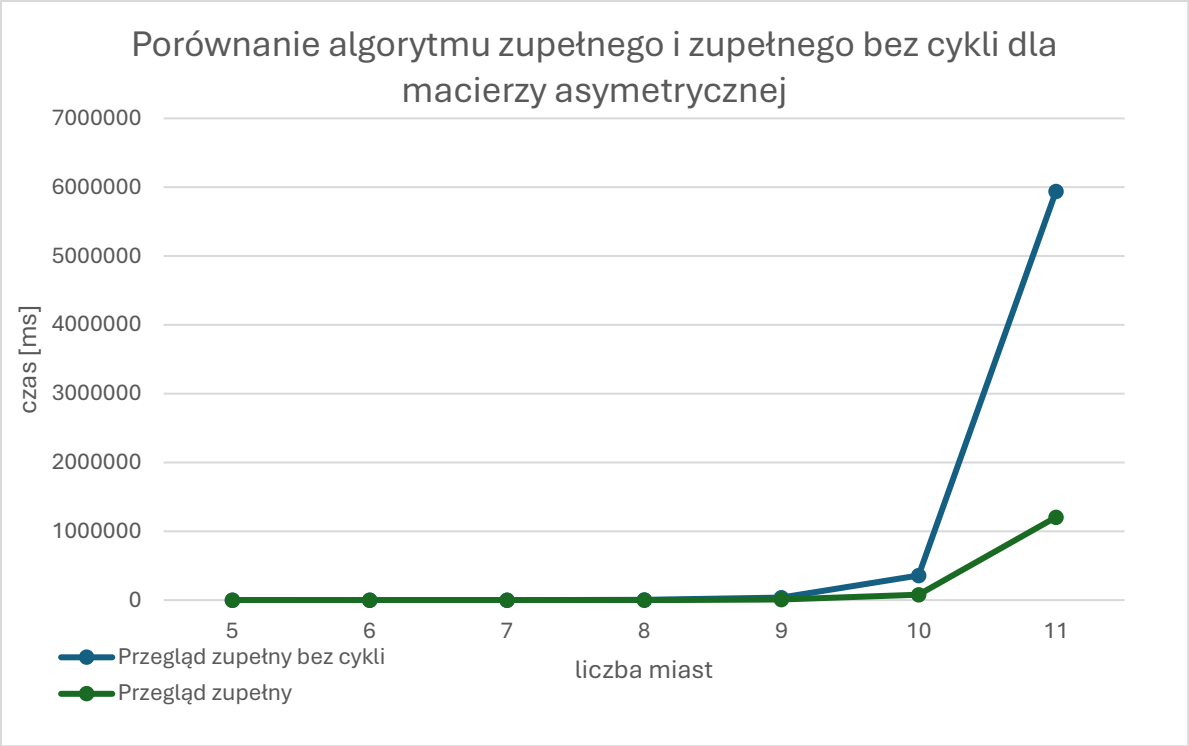
Tabela 5 Tabela porównuje czasy wykonania dwóch wariantów algorytmu przeglądu zupełnego: klasycznego oraz bez uwzględniania ścieżek cyklicznych, w rozwiązaniu problemu komiwojażera (TSP) dla różnych liczb miast (n) w przypadku macierzy symetrycznych.

	Asymetryczne	
	Przegląd zupełny	Przegląd zupełny bez cykli
n	**Czas [ms]**	
2	0	2
3	0	3
4	2	9
5	7	31
6	45	151
7	201	793
8	1400	5757
9	8334	35410
10	76990	359804
11	1204600	5939676

Źródło: Opracowanie własne

Dla małych wartości n, różnica w czasach działania obu algorytmów jest niewielka, ale zaczyna się stopniowo zwiększać wraz ze wzrostem liczby miast. Dla n = 2 lub 3, różnice czasowe są marginalne (różnica wynosi 2-3 ms), ale dla większych instancji problemu (n = 10 i n = 11), czas przeglądu bez cykli dramatycznie rośnie.

Przykładowo, dla n = 10 klasyczny algorytm przeglądu zupełnego potrzebuje około 77 sekund, podczas gdy algorytm bez cykli wymaga już ponad 359 sekund. Dla n = 11 czas działania klasycznego algorytmu to około 1,2 miliona milisekund, natomiast dla przeglądu bez cykli czas ten wzrasta ponad pięciokrotnie, osiągając prawie 6 milionów milisekund.



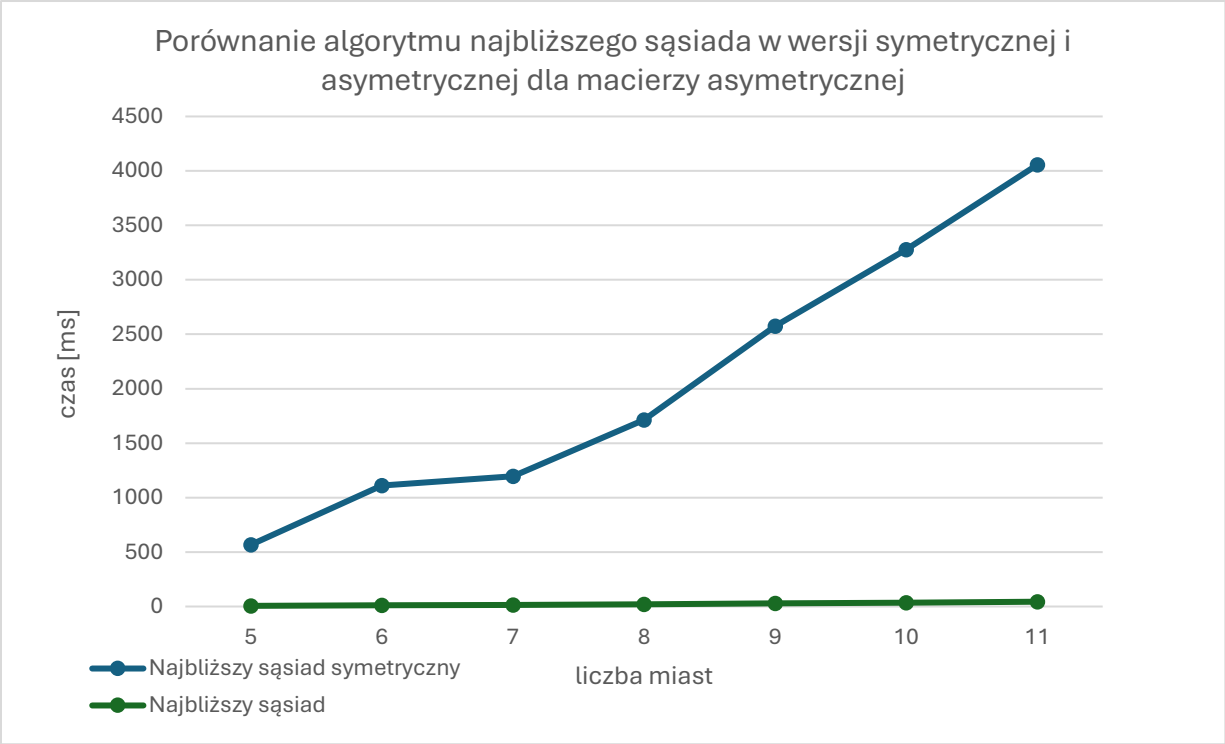
Rysunek 6 Porównanie czasów wykonania dwóch wariantów algorytmu przeglądu zupełnego: klasycznego oraz bez uwzględniania ścieżek cyklicznych, w rozwiązaniu problemu komiwojażera (TSP) dla różnych liczb miast (n) w przypadku macierzy asymetrycznych.
Źródło: Opracowanie własne

Tabela 6 Tabela porównuje czasy wykonania dwóch wariantów algorytmu najbliższego sąsiada dla różnych wielkości instancji problemu komiwojażera (TSP), gdzie n oznacza liczbę miast: klasycznego algorytmu najbliższego sąsiada oraz symetrycznego.

	Asymetryczne	
	Najbliższy sąsiad	Najbliższy sąsiad symetryczny
n	**Czas [ms]**	
2	1	101
3	1	122
4	3	281
5	6	567
6	12	1111
7	13	1195
8	20	1713
9	28	2575
10	36	3276
11	44	4055

Źródło: Opracowanie własne

Wyniki są bardzo zbliżone do tych dla tabeli nr. 3 dla macierzy symetrycznej.



Rysunek 7 Porównanie czasów wykonania dwóch wariantów algorytmu najbliższego sąsiada dla różnych wielkości instancji problemu komiwojażera (TSP), gdzie n oznacza liczbę miast: klasycznego algorytmu najbliższego sąsiada oraz symetrycznego.
Źródło: Opracowanie własne

5. Wnioski

- Na podstawie rysunku nr. 2 możemy wywnioskować, że algorytm przeglądu zupełnego zapewniając najwyższą dokładność płaci czasem obliczeń, natomiast algorytm najbliższego sąsiada oferuje szybsze wyniki, choć kosztem optymalności. Algorytm losowy, ze względu na swoją nieprzewidywalność, okazuje się najmniej efektywnym podejściem w analizowanych przypadkach.
- Porównując wyniki w tabeli nr. 2 i rysunku nr. 3 możemy zauważyć, że algorytm przeglądu zupełnego bez ścieżek cyklicznych, które potencjalnie powinien mieć mniejszą liczbę przeszukiwanych rozwiązań, ma większą złożoność obliczeniową niż jego klasyczna wersja. Wynika to prawdopodobnie z dodatkowych operacji (wyliczenie permutacji bez cyklu), które zwiększają czas działania wraz ze wzrostem liczby miast. Ostatecznie, algorytm bez cykli staje się mniej efektywny czasowo, szczególnie dla dużych instancji problemu, co ogranicza jego zastosowanie do niewielkich zbiorów danych.
- Analizując rysunek nr. 4 da się zauważyć widoczną przewagę algorytmu w wariancie symetrycznym nad asymetrycznym. Daje on lepsze wyniki, jednak kosztem dużego czasu obliczeń. Nie do końca zrozumiałe jest, dlaczego występuje taki rezultat. Jednak można by podejrzewać, że jest to kwestia, iż wariant asymetryczny sprawdza od razu dwa razy więcej ścieżek.
- Porównując rysunek nr. 2 i nr. 5, da się zauważyć bardzo podobny charakter wyników dla algorytmów przeglądu zupełnego i najbliższego sąsiada. Pokazuje to stabilność obu tych algorytmów. Dla algorytmu losowego nie da się wyznaczyć przewidywalnego kształtu, jednak widać, że czas widocznie rośnie wraz z liczbą miast. Może to wynikać z implementacji algorytmu i nie optymalnych warunków podanych do zakończenia pętli.

Podsumowując:

Algorytm przeglądu zupełnego zapewnia optymalne rozwiązania, ale jego wykładniczo rosnący czas obliczeń ogranicza zastosowanie do małych instancji problemu.

Algorytm najbliższego sąsiada stanowi korzystny kompromis między jakością rozwiązania a wydajnością czasową, zwłaszcza dla średnich i dużych wartości n. Wariant symetryczny, mimo zwiększonego czasu działania, może być rozważany, gdy symetria problemu jest istotna.

Algorytm losowy ze względu na nieprzewidywalność i długi czas działania przy większych n nie jest rekomendowany do praktycznych zastosowań w rozwiązywaniu problemu komiwojażera.

6. Kod źródłowy

Zostanie załączony wraz ze sprawozdaniem.

Bibliografia:

1. http://algorytmy.ency.pl/artykul/problem_komiwojazera
2. http://algorytmy.ency.pl/artykul/algorytm_najblizszego_sasiada
3. https://eduinf.waw.pl/inf/alg/001_search/0140.php
4. https://tonicanada.medium.com/introduction-to-the-travelling-salesman-problem-5ace44932cb5
5. **"Introduction to Algorithms"** - Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein
6. *"The Traveling Salesman Problem and Its Variations"* – **Gutin, G., & Punnen, A. P.** (Eds.). (2002)